COSC 581 Programming Assignment 1 Write-up

Name: Sarah Huang
Objective: Implement a quicksort algorithm that uses the median-of-medians algorithm to find the pivot value.

Partition Function

The partition function organizes the elements so that values less than the pivot are to its left and values greater than the pivot are to its right. It swaps the values at the pivot and last index, so the pivot value isn't compared with itself. The start index is also stored in a new temporary storage variable which will be used to keep track of where the pivot goes in the end. It iterates through the array to check if the element is smaller than the pivot value. If the element is smaller than the pivot, it swaps with the value at the temp index and the temp index gets incremented. The process keeps going until the pivot is swapped from the back of the array to the temp index. The function returns what the final pivot index is.

Median-of-Medians Function & Finding the Median

The median-of-median function recursively finds the median from a calculated list of medians. Given a r value, the entire array is split into groups of size r. The median of each group is calculated by using the partition function to find a pivot and search based on that pivot location. If the pivot index is the median position, we found the median. If the pivot is before the median, search the right side of the pivot. If the pivot is after the median, search the left side. Therefore, we find the median without directly sorting the array.

Results

Duration (in seconds) to sort N numbers if splitting elements into r size groups					
	r = 3	r = 5	r = 7	r = 9	r = 11
N = 10	0.00051164627075	0.0	0.0	0.0	0.0
N = 100	0.00131773948669	0.0005478858	0.00052928924	0.0	0.0
N = 1,000	0.00677013397216	0.0048410892	0.00454568862	0.00436925888	0.00363326072
N = 10,000	0.08632421493530	0.0645577907	0.05803322792	0.05574440956	0.05074834823
N = 100,000	1.12770295143127	0.8263680934	0.71662521362	0.68913125991	0.631401062011
N = 1,000,000	17.4257023334503	12.934358119	11.12964963912	10.0026352405	9.68350219726

Notice how r = 11 has the shortest time to complete compared to the other r values. In fact, the greater the r value, the faster the sorting time. In other words, splitting elements into groups of a bigger size in median-of-medians is more time efficient than having more groups with smaller sizes.